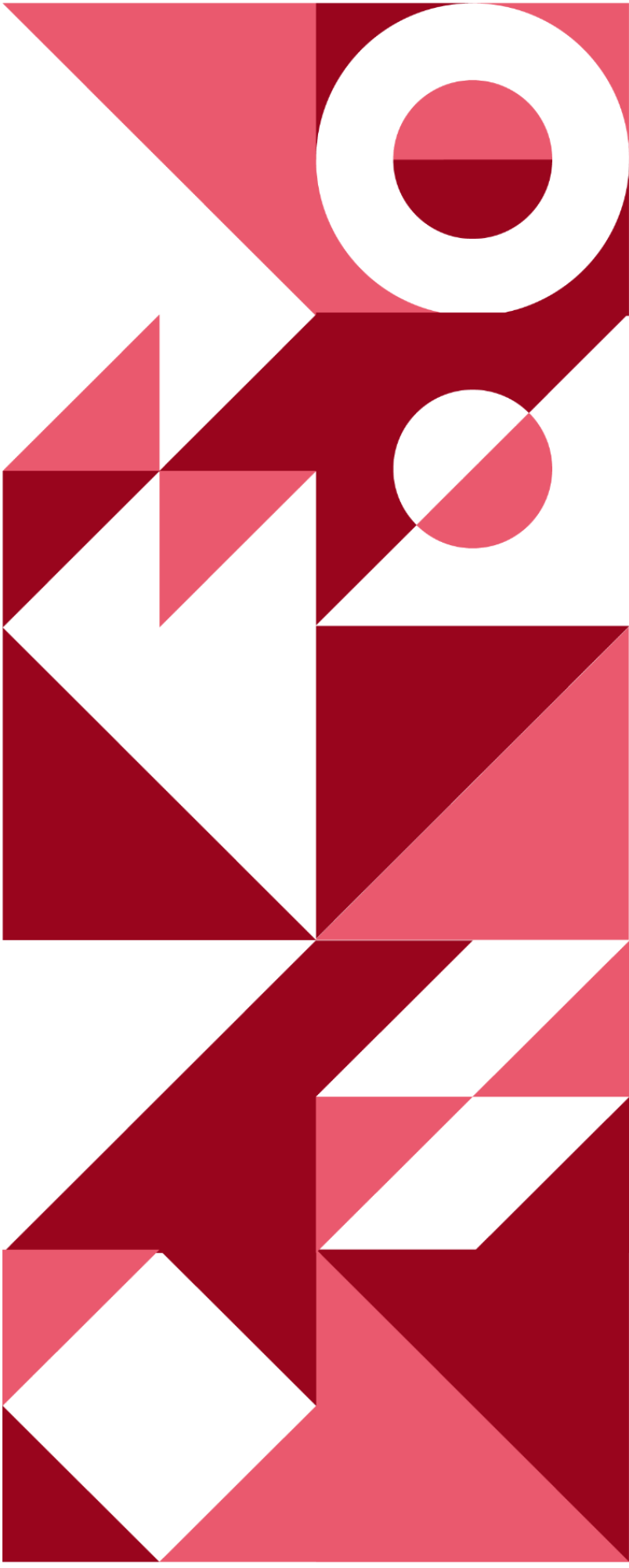


Serum v4

Audit

Presented by:

OtterSec contact@osec.io

William Wang defund@osec.io

Robert Chen notdeghost@osec.io

Table of Contents

Table of Contents **1**

01 | Executive Summary **3**

Overview 3

Key Findings 3

02 | Scope **4**

03 | Procedure **5**

04 | Findings **6**

Proofs of Concept 6

05 | Vulnerabilities **8**

OS-SRM-ADV-00 [Critical] [Resolved]: Accounts are not necessarily rent-exempt 9

OS-SRM-ADV-01 [High] [Resolved]: Missing ownership checks for vaults 11

OS-SRM-ADV-02 [High] [Resolved]: Event queue denial of service 13

OS-SRM-ADV-03 [Medium] [Resolved]: Fee tier spoofing 14

OS-SRM-ADV-04 [Medium] [Resolved]: Self-trading is broken 16

OS-SRM-ADV-05 [Medium] [Resolved]: Order book arithmetic overflow 18

OS-SRM-ADV-06 [Medium] [Resolved]: Account tag collision 20

OS-SRM-ADV-07 [Low] [Resolved]: Markets cannot be closed 21

06 | General Findings **22**

OS-SRM-SUG-00: Unchecked arithmetic 23

OS-SRM-SUG-01 [Resolved]: Incorrect order book spread 24

OS-SRM-SUG-02 [Resolved]: Incorrect slab capacity 26

OS-SRM-SUG-03 [Resolved]: Slab mutates on error 27

OS-SRM-SUG-04 [Resolved]: Minimum order size should be positive 28

OS-SRM-SUG-05: Uninitialized account tag is unsafe 29

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-06: Inconsistent account tag serialization 30

OS-SRM-SUG-07: Missing ownership check for market 31

OS-SRM-SUG-08: Redundant account checks 32

OS-SRM-SUG-09: Outdated Cargo dependency 34

OS-SRM-SUG-10: Market creation allows duplicate accounts 35

07 | Appendix 36

Appendix A: Program Files 36

Appendix B: Implementation Security Checklist 38

Appendix C: Proofs of Concept 40

Appendix D: Vulnerability Rating Scale 41

© 2022 OtterSec LLC. All Rights Reserved.

01 | Executive Summary

Overview

Serum engaged OtterSec to perform an assessment of the `dex-v4` and `agnostic-orderbook` programs. This assessment was conducted between April 4th and May 6th, with a focus on the following:

- Ensure integrity of the programs at an implementation and design level.
- Analyze the program attack surface and provide meaningful recommendations to mitigate future vulnerabilities.

Critical vulnerabilities were communicated to the team prior to the delivery of the report to speed up remediation. After delivering our audit report, we worked closely with the Serum team to streamline patches and confirm remediation.

Key Findings

The following is a summary of the major findings in this audit.

- 19 findings total, 8 of which we classify as vulnerabilities
- 3 vulnerabilities which could lead to loss of funds
    - OS-SRM-ADV-00: Accounts are not necessarily rent-exempt
    - OS-SRM-ADV-01: Vaults are not required to be owned by the SPL token program
    - OS-SRM-ADV-02: User accounts are not explicitly closed

We also observed the following:

- The codebase was well-structured and easy to understand.
- The team was very knowledgeable and responsive to our feedback.
- Test coverage was somewhat lacking, and many findings were related to standard functionality.
- The most severe vulnerabilities were related to issues with the Solana runtime and SPL token program.

As part of this audit, we implemented proofs of concept for each vulnerability. They are available at https://osec.io/pocs/serum and a full list can be found in Appendix C.

© 2022 OtterSec LLC. All Rights Reserved.

02 | Scope

We received the program from Serum and began the audit April 4th. The source code was delivered to us in a git repository at https://github.com/Bonfida/agnostic-orderbook and https://github.com/bonfida/dex-v4. This audit was performed against commits 176b410 and 748aaf9 respectively.

A brief description of the two programs is as follows. A full list of program files and hashes can be found in Appendix A.

Name	Description
agnostic-orderbook (Aob)	Generic implementation of an order book
dex-v4 (Dex)	Program wrapper around Aob which supports user accounts, payments, and other Serum-specific logic

© 2022 OtterSec LLC. All Rights Reserved.

03 | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an onchain program. In other words, there is no way to steal tokens or deny service, ignoring any Solana specific quirks such as account ownership issues. An example of a design vulnerability would be an onchain oracle which could be manipulated by flash loans or large deposits.

On the other hand, auditing the implementation of the program requires a deep understanding of Solana's execution model. Some common implementation vulnerabilities include account ownership issues, arithmetic overflows, and rounding bugs. For a non-exhaustive list of security issues we check for, see Appendix B. A repeated issue we observed was missing account ownership checks for SPL token accounts (OS-SRM-ADV-01, OS-SRM-ADV-03).

Implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program. For example, Aob and Dex fulfill trades in two stages: immediately and through an event queue. We found that self-trades were not properly handled in the latter (OS-SRM-ADV-04).

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach any target in a team of two. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

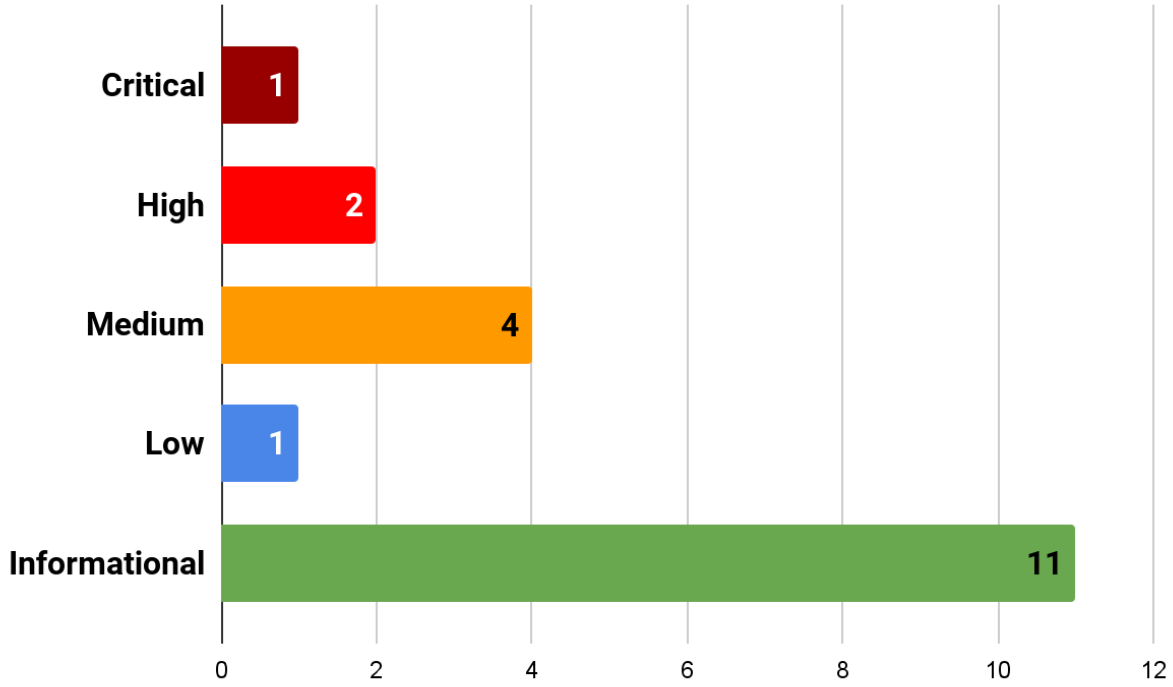
© 2022 OtterSec LLC. All Rights Reserved.

04 | Findings

In total, we report 19 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings don't have an immediate impact but will help mitigate future vulnerabilities.

The following chart displays our findings by severity.



Proofs of Concept

For each vulnerability, we implemented proofs of concept to demonstrate exploitability. They are available at https://osec.io/pocs/serum and a full list can be found in Appendix C. In this audit, we wrote proofs of concept by modifying the program's existing test framework. They are stored as patch files, meant to be applied at the target commit.

Proofs of concept also function as an effective regression test. We recommend integrating them as part of a comprehensive test suite.

© 2022 OtterSec LLC. All Rights Reserved.

To run a proof of concept, use the provided script:

```
./run.sh <directory name>
```

For example, to run OS-SRM-ADV-00:

```
./run.sh os-srm-adv-00
```

© 2022 OtterSec LLC. All Rights Reserved.

05 | Vulnerabilities

Here we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

Rating criterion can be found in Appendix E.

ID	Severity	Status	Description
OS-SRM-ADV-00	**Critical**	**Resolved**	Accounts are not necessarily rent-exempt, leading to loss of funds
OS-SRM-ADV-01	**High**	**Resolved**	Vaults are not required to be owned by the SPL token program, leading to loss of funds in a maliciously created market
OS-SRM-ADV-02	**High**	**Resolved**	User accounts are not explicitly closed, leading to loss of funds and denial of service
OS-SRM-ADV-03	**Medium**	**Resolved**	Discount token accounts are not required to be owned by the SPL token program, leading to fee tier spoofing
OS-SRM-ADV-04	**Medium**	**Resolved**	Self-trading is broken, leading to loss of funds
OS-SRM-ADV-05	**Medium**	**Resolved**	Order book arithmetic overflow results in unexpected behavior
OS-SRM-ADV-06	**Medium**	**Resolved**	Account tag collisions between Aob and Dex results in unexpected behavior
OS-SRM-ADV-07	**Low**	**Resolved**	Markets cannot be closed

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-00 [Critical] [Resolved]: Accounts are not necessarily rent-exempt

Description

Aob's `create_market` instruction assumes the `market`, `event_queue`, `bids`, and `asks` accounts have been created externally, and similarly for Dex's `create_market` instruction.

If one of these accounts is not rent-exempt, it will eventually run out of lamports and be purged. When this happens, the exchange will stop functioning.

// agnostic-orderbook/program/src/processor/create_market.rs:L43-L57

```
pub struct Accounts<'a, T> {
    #[allow(missing_docs)]
    #[cons(writable)]
    pub market: &'a T,
    #[allow(missing_docs)]
    #[cons(writable)]
    pub event_queue: &'a T,
    #[allow(missing_docs)]
    #[cons(writable)]
    pub bids: &'a T,
    #[allow(missing_docs)]
    #[cons(writable)]
    pub asks: &'a T,
}
```

Moreover, the purged account may be recreated at the same address, then used in a new exchange. This results in “duplicate references,” which an attacker can leverage.

Proof of Concept

Suppose tokens A and B are roughly equal in value.

1. An attacker initializes an exchange with base A and quote B. The `bids` and `asks` accounts are not rent-exempt, i.e. they hold very few lamports.
2. Normal users interact with the exchange, and their locked tokens are stored in the vaults. Suppose the base vault has 100A.
3. After some epochs, `bids` and `asks` have zero lamports and are purged. At this point, the exchange is dysfunctional.
4. The attacker recreates `bids` and `asks` at the same addresses.

© 2022 OtterSec LLC. All Rights Reserved.

5. The attacker uses `bids` and `asks` to initialize a dummy exchange with base C and quote D. These may be dummy tokens that the attacker can mint arbitrarily. At this point the legitimate exchange is functional, albeit with a wiped order book.
6. The attacker posts an ask of 100C for 1D on the dummy exchange. On the legitimate exchange, this is interpreted as 100A for 1B. Note the attacker did not need to transfer 100A to the legitimate exchange's base vault.
7. The attacker posts a bid of 100A for 1B on the legitimate exchange. This is matched and the attacker receives 100A, minus fees.

To conclude, the attacker was able to pay 1B for 100A — arbitrarily better than the market rate.

Remediation

All uninitialized accounts should be created within the program so that they are rent-exempt. This is what Dex's `initialize_account` instruction already does for `UserAccount`.

```
// dex-v4/program/src/processor/initialize_account.rs:L111-L119

let lamports = Rent::get()?.minimum_balance(space as usize);

let allocate_account = create_account(
    accounts.fee_payer.key,
    accounts.user.key,
    lamports,
    space,
    program_id,
);
```

Another option is to check that the provided accounts are rent-exempt.

Patch

Added rent exemption checks in #61 and #56.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-01 [High] [Resolved]: Missing ownership checks for vaults

Dex's `create_market` instruction requires the token authority of the base and quote vaults to be the `market_signer` PDA. However, it does not check that the `base_vault` and `quote_vault` accounts are in fact owned by the SPL token program.

```
// dex-v4/program/src/processor/create_market.rs:L172-L180

let acc = spl_token::state::Account::unpack(&account.data.borrow())?;
if &acc.owner != market_signer {
    msg!("The vault account should be owned by the market signer");
    return Err(ProgramError::InvalidArgument);
}
if acc.close_authority.is_some() || acc.delegate.is_some() {
    msg!("Invalid vault account provided");
    return Err(ProgramError::InvalidArgument);
}
```

When initializing the exchange, an attacker can spoof `base_vault` and `quote_vault` accounts that bypass the token authority check. Later, they can close the accounts and recreate them as valid token wallets, owned by the SPL token program. This allows them to obtain token authority over the base and quote vaults.

Proof of Concept

1. An attacker forges `base_vault` and `quote_vault` accounts. When interpreted as an SPL token wallet, the token authority should be the `market_signer` PDA.
2. An attacker initializes an exchange with `base_vault` and `quote_vault`. At this point, the exchange is dysfunctional.
3. The attacker closes `base_vault` and `quote_vault`, then recreates them at the same addresses with the SPL token program. At this point, the exchange is partially functional: the `settle` and `swap` instructions will fail, but the `new_order` instruction will work.
4. The attacker withdraws from `base_vault` and `quote_vault`, assuming they control the token authority.

To conclude, the attacker was able to drain locked tokens from the exchange's vaults.

Remediation

Dex's `create_market` instruction already checks that it owns the `market` and `orderbook` accounts.

© 2022 OtterSec LLC. All Rights Reserved.

```
// dex-v4/program/src/processor/create_market.rs:L88-L89
```

```
check_account_owner(a.market, program_id, DexError::InvalidStateAccountOwner)?;
check_account_owner(a.orderbook, program_id, DexError::InvalidStateAccountOwner)?;
```

Additional checks for `base_vault` and `quote_vault` accounts should be included here.

```
check_account_owner(a.base_vault, &spl_token::ID,
DexError::InvalidBaseVaultAccount)?;
check_account_owner(a.quote_vault, &spl_token::ID,
DexError::InvalidQuoteVaultAccount)?;
```

Patch

Fixed in #51.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-02 [High] [Resolved]: Event queue denial of service

Description

Dex's `close_account` instruction transfers lamports out of the `user` account, but it may not be immediately purged. During this grace period, it is possible to file a new order with `user`. If the order is matched after `user` is purged, it will be impossible to consume the emitted event. In particular, Dex's `consume_events` instruction will return a `MissingUserAccount` error.

```
// dex-v4/program/src/processor/consume_events.rs:L163-L167

let maker_info =
    CallBackInfo::deserialize(&mut (&maker_callback_info as &[u8])).unwrap();
let maker_account_info = &accounts[accounts
    .binary_search_by_key(&maker_info.user_account, |k| *k.key)
    .map_err(|_| DexError::MissingUserAccount)?];
```

Subsequent `Fill` and `Out` events pushed to the event queue will be inaccessible, resulting in loss of funds. Once the event queue is filled, the market will be rendered dysfunctional, resulting in denial of service.

Proof of Concept

1. An attacker creates two user accounts on an existing exchange.
2. In a single transaction, the attacker closes the first user account, then files a new order with the first user.
3. After the first user account is purged, the attacker matches the order with the second user. A `Fill` event is pushed to the event queue. It is impossible to dereference the callback information.

To conclude, the attacker was able to freeze the event queue.

Remediation

Dex's `close_account` instruction should additionally reset the account tag of `user`.

```
user_account.header.tag = AccountTag::Closed as u64;
```

Patch

Fixed in #56.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-03 [Medium] [Resolved]: Fee tier spoofing

Description

Dex's `new_order` instruction incorrectly checks that the `discount_token_account` account is owned by the `user_owner` address, rather than the SPL token program. This allows an attacker to obtain arbitrary fee tiers without the requisite token balance.

```
// dex-v4/program/src/processor/new_order.rs:L171-L177

if let Some(discount_account) = a.discount_token_account {
    check_account_owner(
        discount_account,
        a.user_owner.key,
        DexError::InvalidStateAccountOwner,
    )?;
}
```

The user_owner key should be compared against the parsed token authority, not the account's intrinsic owner. Note this is already done in the `FeeTier::get` method, along with token mint validation.

```
// dex-v4/program/src/state.rs:L318-L330

let parsed_token_account =
spl_token::state::Account::unpack(&account.data.borrow())?;
if &parsed_token_account.owner != expected_owner {
    msg!("The discount token account must share its owner with the user account.");
    return Err(ProgramError::InvalidArgument);
}
let (srm_held, msrm_held) = match parsed_token_account.mint {
    a if a == MSRM_MINT => (0, parsed_token_account.amount),
    a if a == SRM_MINT => (parsed_token_account.amount, 0),
    _ => {
        msg!("Invalid mint for discount token acccount.");
        return Err(ProgramError::InvalidArgument);
    }
};
```

Proof of Concept

1. An attacker forges a `discount_token_account` account which is owned by their `user_owner` address rather than the SPL token program. When interpreted as an SPL

© 2022 OtterSec LLC. All Rights Reserved.

token wallet, the token authority is `user_owner`, the mint is MSRM, and the balance is 1.

2. The attacker files a new order using `discount_token_account`, and the `MSrm` fee tier is applied.

To conclude, the attacker was able to use the `MSrm` fee tier without proving ownership of MSRM.

Remediation

The `user_owner` key should be replaced with the SPL token program ID.

```
if let Some(discount_account) = a.discount_token_account {
    check_account_owner(
        discount_account,
        &spl_token::ID,
        DexError::InvalidStateAccountOwner,
    )?;
}
```

Patch

Fixed in #49 and 47025e3.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-04 [Medium] [Resolved]: Self-trading is broken

Description

Dex's `consume_events` instruction includes special logic for self-trade `Fill` events.

```
// dex-v4/program/src/processor/consume_events.rs:L170-L300

if taker_info.user_account == maker_info.user_account {
    ...
} else {
    ...
}
```

It correctly increases the maker's rebate.

```
// dex-v4/program/src/processor/consume_events.rs:L171-L176

let maker_rebate = taker_fee_tier.maker_rebate(quote_size);
maker_account.header.quote_token_free = maker_account
    .header
    .quote_token_free
    .checked_add(maker_rebate)
    .unwrap();
```

It also correctly decreases the maker's locked base/quote balance.

```
// dex-v4/program/src/processor/consume_events.rs:L194-L209

match taker_side {
    Side::Bid => {
        maker_account.header.base_token_locked = maker_account
            .header
            .base_token_locked
            .checked_sub(base_size)
            .unwrap();
    }
    Side::Ask => {
        maker_account.header.quote_token_locked = maker_account
            .header
            .quote_token_locked
            .checked_sub(quote_size)
            .unwrap();
    }
};
```

© 2022 OtterSec LLC. All Rights Reserved.

However, it fails to increase the free base/quote balance, resulting in loss of funds. We note that once this is included, the logic is equivalent to what is done for normal trades. Indeed, a self-trade `Fill` event is only emitted if an order's self-trade behavior is `DecrementTake`., which ignores the identities of maker and taker.

// agnostic-orderbook/program/src/orderbook.rs:L159-L206

```
if self_trade_behavior != SelfTradeBehavior::DecrementTake {
    let order_would_self_trade = &callback_info[..callback_id_len]
        == (&self
            .get_tree(side.opposite())
            .get_callback_info(best_bo_ref.callback_info_pt as
usize)[..callback_id_len]
            as &[u8]);
    if order_would_self_trade {
        ...
    }
}
```

Proof of Concept

1. A user posts an ask of 1A for 1B on the exchange.
2. The user files a bid of 1A for 1B with the `DecrementTake` self-trade behavior. The order is matched, a `Fill` event is pushed to the event queue, and the user receives 1A minus fees.
3. When the `Fill` event is consumed, the user's free quote balance is not incremented.

To conclude, the user lost approximately 1B from the self-trade.

Remediation

Dex's `consume_events` instruction does not need to handle self-trades differently. The entire if branch should be removed.

Patch

Fixed in #55.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-05 [Medium] [Resolved]: Order book arithmetic overflow

Description

Aob's fixed point operations can overflow, resulting in unexpected behavior. In particular, `fp32_div` and `fp32_mul` may overflow while casting down to u64.

// agnostic-orderbook/program/src/utils.rs:L76-L84

```
/// a is fp0, b is fp32 and result is a/b fp0
pub(crate) fn fp32_div(a: u64, b_fp32: u64) -> u64 {
    (((a as u128) << 32) / (b_fp32 as u128)) as u64
}

/// a is fp0, b is fp32 and result is a*b fp0
pub(crate) fn fp32_mul(a: u64, b_fp32: u64) -> u64 {
    (((a as u128) * (b_fp32 as u128)) >> 32) as u64
}
```

As an example, consider the `OrderBookState::new_order` method.

// agnostic-orderbook/program/src/orderbook.rs:L255-L258

```
let base_qty_to_post = std::cmp::min(
    fp32_div(quote_qty_remaining, limit_price),
    base_qty_remaining,
);
```

Notice that `fp32_div` may overflow when `limit_price` is less than one. If the result is smaller than `base_qty_remaining`, this will unexpectedly constrain `base_qty_to_post`.

Remediation

Aob's fixed point operations should be rewritten with checked arithmetic and casting, identical to what Dex already uses.

// dex-v4/program/src/utils.rs:L39-L60

```
/// a is fp0, b is fp32 and result is a/b fp0
pub(crate) fn fp32_div(a: u64, b_fp32: u64) -> Option<u64> {
    ((a as u128) << 32)
        .checked_div(b_fp32 as u128)
        .and_then(safe_downcast)
```

© 2022 OtterSec LLC. All Rights Reserved.

```
}

/// a is fp0, b is fp32 and result is a*b fp0
pub(crate) fn fp32_mul(a: u64, b_fp32: u64) -> Option<u64> {
    (a as u128)
        .checked_mul(b_fp32 as u128)
        .and_then(|e| safe_downcast(e >> 32))
}

fn safe_downcast(n: u128) -> Option<u64> {
    static BOUND: u128 = u64::MAX as u128;
    if n > BOUND {
        None
    } else {
        Some(n as u64)
    }
}
```

Patch

Fixed in #51.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-06 [Medium] [Resolved]: Account tag collision

Description

Aob uses the `AccountTag` enum to distinguish between account types.

```
// agnostic-orderbook/program/src/state.rs:L26-L32

pub enum AccountTag {
    Uninitialized,
    Market,
    EventQueue,
    Bids,
    Asks,
}
```

However, Dex also employs this strategy.

```
// dex-v4/program/src/state.rs:L18-L23

pub enum AccountTag {
    Uninitialized,
    DexState,
    UserAccount,
    Closed,
}
```

Since the crates use overlapping integer values during serialization, this results in possible type confusion between

- `Market` and `DexState`
- `EventQueue` and `UserAccount`
- `Bids` and `Closed`

Remediation

Aob and Dex should use disjoint enum ranges or add an additional field to distinguish between crates.

Patch

Modified to use disjoint enum ranges in #54.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-ADV-07 [Low] [Resolved]: Markets cannot be closed

Description

Dex's `close_market` instruction attempts to transfer lamports out of the `base_vault` and `quote_vault` accounts, which the program does not own. This is disallowed by the Solana runtime.

```
// dex-v4/program/src/processor/close_market.rs:L138-L148

let mut market_lamports = accounts.market.lamports.borrow_mut();
let mut base_vault_lamports = accounts.base_vault.lamports.borrow_mut();
let mut quote_vault_lamports = accounts.quote_vault.lamports.borrow_mut();

let mut target_lamports = accounts.target_lamports_account.lamports.borrow_mut();

**target_lamports += **market_lamports + **base_vault_lamports +
**quote_vault_lamports;

**market_lamports = 0;
**base_vault_lamports = 0;
**quote_vault_lamports = 0;
```

Remediation

Dex's `close_market` instruction should invoke `spl_token::instruction::close_account` to close the vaults.

Patch

Fixed in #52.

© 2022 OtterSec LLC. All Rights Reserved.

06 | General Findings

Here we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they do represent anti-patterns and could introduce a vulnerability in the future.

ID	Description
OS-SRM-SUG-00	Unchecked arithmetic
OS-SRM-SUG-01	Incorrect order book spread
OS-SRM-SUG-02	Incorrect slab capacity
OS-SRM-SUG-03	Slab mutates on error
OS-SRM-SUG-04	Minimum order size should be positive
OS-SRM-SUG-05	Uninitialized account tag is unsafe
OS-SRM-SUG-06	Inconsistent account tag serialization
OS-SRM-SUG-07	Missing ownership check for market
OS-SRM-SUG-08	Redundant account checks
OS-SRM-SUG-09	Outdated Cargo dependency
OS-SRM-SUG-10	Market creation allows duplicate accounts

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-00: Unchecked arithmetic

Description

We found many instances of unchecked arithmetic which, while not posing a security risk, lead to unexpected behavior. For example, Dex's `initialize_account` instruction uses unchecked arithmetic when calculating buffer size.

```
// dex-v4/program/src/processor/initialize_account.rs:L109

let space = (USER_ACCOUNT_HEADER_LEN as u64) + max_orders * (Order::LEN as u64);
```

It is possible for a client to specify `max_order` such that the theoretical space required is too large to allocate, yet the overflow result is reasonably small. The instruction would succeed, creating a discrepancy between the client interface and the on-chain program.

Remediation

Use wrapping methods (`wrapping_add`, `wrapping_mul`, etc.) to explicitly allow overflows. Then in `Cargo.toml`, append the following:

```
[profile.release]
overflow-checks = true
```

This will enable integer overflow checks by default.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-01 [Resolved]: Incorrect order book spread

Description

Aob's `OrderBookState::get_spread` method incorrectly uses `find_max` to obtain the best ask price.

```
// agnostic-orderbook/program/src/orderbook.rs:L64-L74

pub fn get_spread(&self) -> (Option<u64>, Option<u64>) {
    let best_bid_price = self
        .bids
        .find_max()
        .map(|h| self.bids.get_node(h).unwrap().as_leaf().unwrap().price());
    let best_ask_price = self
        .asks
        .find_max()
        .map(|h| self.asks.get_node(h).unwrap().as_leaf().unwrap().price());
    (best_bid_price, best_ask_price)
}
```

Note that this method is not directly used in Aob and Dex, so there is no immediate security impact. However, the logic is exposed as a utility function.

```
// agnostic-orderbook/program/src/utils.rs:L60-L74

pub fn get_spread<'ob>(
    market_state_account: &AccountInfo<'ob>,
    bids_account: &AccountInfo<'ob>,
    asks_account: &AccountInfo<'ob>,
) -> (Option<u64>, Option<u64>) {
    let market_state = MarketState::get(market_state_account).unwrap();
    let orderbook = OrderBookState::new_safe(
        bids_account,
        asks_account,
        market_state.callback_info_len as usize,
        market_state.callback_id_len as usize,
    )
    .unwrap();
    orderbook.get_spread()
}
```

Other programs which use Aob as a standalone library may call `get_spread`, which poses a security risk.

© 2022 OtterSec LLC. All Rights Reserved.

Remediation

The best ask price is the smallest quantity, so Aob's `OrderBookState::get_spread` method should call `find_min` instead.

Patch

Fixed in bab9dfb.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-02 [Resolved]: Incorrect slab capacity

Description

Aob's `Slab` struct implements a crit-bit tree, where each leaf node is an order. It calculates how many orders it can hold based on how many nodes it can hold in total.

// agnostic-orderbook/program/src/critbit.rs:L299-L302

```
fn capacity(&self) -> u64 {
    ((self.buffer.borrow().len() - PADDED_SLAB_HEADER_LEN)
        / (2 * SLOT_SIZE + self.callback_info_len)) as u64
}
```

Notice that it assumes an `N`-leaf crit-bit tree requires `2N` nodes in total.

// agnostic-orderbook/program/src/critbit.rs:L383-L385

```
if self.header.bump_index as usize == 2 * self.capacity() as usize {
    return Err(std::io::ErrorKind::UnexpectedEof.into());
}
```

Remediation

An `N`-leaf crit-bit tree only requires `N - 1` inner nodes, and hence `2N - 1` nodes in total. Modifying `Slab::capacity` to reflect this would yield a more space-efficient implementation. The following logic should also be updated.

// agnostic-orderbook/program/src/critbit.rs:L383-L385

```
let order_capacity = (asks_account.data.borrow().len() - PADDED_SLAB_HEADER_LEN)
    / (SLOT_SIZE * 2 + callback_info_len);

let asks_callback_memory_offset = PADDED_SLAB_HEADER_LEN + 2 * order_capacity *
SLOT_SIZE;
...
let bids_order_capacity = (bids_account.data.borrow().len() -
PADDED_SLAB_HEADER_LEN)
    / (SLOT_SIZE * 2 + callback_info_len);
let bids_callback_memory_offset =
    PADDED_SLAB_HEADER_LEN + 2 * bids_order_capacity * SLOT_SIZE;
```

Patch

Fixed in #52 and #58.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-03 [Resolved]: Slab mutates on error

Description

When Aob's `Slab::insert_leaf` method is called, it will most likely allocate a leaf node followed by an inner node.

```
// agnostic-orderbook/program/src/critbit.rs:L641-L647

let new_leaf_handle = self
    .insert_node(new_leaf_node)
    .map_err(|_| AoError::SlabOutOfSpace)?;

let new_root_node_handle = self
    .allocate(&NodeTag::Inner)
    .map_err(|_| AoError::SlabOutOfSpace)?;
```

Suppose the first allocation is successful, but the second is not. The method will return an error, which is caught by the OrderBookState::new_order method.

```
// agnostic-orderbook/program/src/orderbook.rs:L280-L282

if let Err(AoError::SlabOutOfSpace) = insert_result {
    // Boot out the least aggressive orders
    msg!("Orderbook is full! booting lest aggressive orders...");
```

However, notice that a newly allocated node, disconnected from the tree, remains. Although this does not pose a serious issue here, modifying state on error is an anti-pattern. In more complex data structures, analogous bugs could result in memory leakage and subsequently denial of service.

Remediation

Aob's `Slab::insert_leaf` method should not allocate space if the insertion will not succeed. This can be checked in advance by examining the total capacity versus the recorded leaf count.

Patch

Fixed in #52.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-04 [Resolved]: Minimum order size should be positive

Description

In Aob, markets are initialized with a `min_base_order_size` quantity. The goal is to prevent small orders from flooding the market.

```
// agnostic-orderbook/program/src/orderbook.rs:L260-L2267

if crossed || !post_allowed || base_qty_to_post < min_base_order_size {
    return Ok(OrderSummary {
        posted_order_id: None,
        total_base_qty: max_base_qty - base_qty_remaining,
        total_quote_qty: max_quote_qty - quote_qty_remaining,
        total_base_qty_posted: 0,
    });
}
```

If `min_base_order_size` is set to zero, the market is technically functional. However, users would be able to post arbitrarily priced orders with zero quantity.

Remediation

Aob's `create_market` instruction should explicitly check that `min_base_order_size` is positive.

Patch

Fixed in #61.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-05: Uninitialized account tag is unsafe

Description

Aob's `close_market` instruction should assign a new tag named `Closed`, rather than the `Uninitialized` tag. Since accounts with zero lamports may not be purged immediately, the program needs to explicitly disallow reusing them when initializing another exchange.

```
// agnostic-orderbook/program/src/processor/close_market.rs:L109

market_state.tag = AccountTag::Uninitialized as u64;
```

The issue stems from the fact that `Uninitialized` serializes to the integer value zero, which already represents a newly created account.

Remediation

Aob's `close_market` instruction should assign a new tag named `Closed`. In general, it should avoid using the `Uninitialized` tag or delete it altogether. The same applies for Dex, whose `create_market` instruction should explicitly compare the buffer's first byte with zero instead.

```
// dex-v4/program/src/processor/create_market.rs:L115-119

if accounts.market.data.borrow()[0] != AccountTag::Uninitialized as u8 {
    // Checking the first byte is sufficient as there is a small number of
AccountTags
    msg!("The market account contains initialized state!");
    return Err(ProgramError::InvalidArgument);
}
```

In both crates, the `AccountTag` enum's documentation should disallow serialization to zero.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-06: Inconsistent account tag serialization

Description

Account tags are serialized to buffers with inconsistent sizing. In Aob, the `MarketState` tag is serialized as u64, while the `EventQueue` and `Slab` tags are serialized as u8. In Dex, the `DexState` and `UserAccount` tags are serialized as u64.

In practice, however, the program assumes account tags fit within a single byte. For example, Aob's `create_market` instruction uses the `utils::check_unitialized` function, which would break if an account tag is serialized to 256.

```
// agnostic-orderbook/program/src/utils.rs:L51-L56

pub(crate) fn check_unitialized(account: &AccountInfo) -> AoResult {
    if account.data.borrow()[0] != 0 {
        return Err(AoError::AlreadyInitialized);
    }
    Ok(())
}
```

Similarly, Dex's create_market instruction only checks the first byte of an account buffer.

```
// dex-v4/program/src/processor/create_market.rs:L115-119

if accounts.market.data.borrow()[0] != AccountTag::Uninitialized as u8 {
    // Checking the first byte is sufficient as there is a small number of
AccountTags
    msg!("The market account contains initialized state!");
    return Err(ProgramError::InvalidArgument);
}
```

Remediation

It would be safer to consistently serialize account tags as u8.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-07: Missing ownership check for market

Description

Aob's `create_market` instruction does not compare the `market` account's owner against the program ID.

```
// agnostic-orderbook/program/src/processor/create_market.rs:L74-L80
```

```
check_account_owner(
    self.event_queue,
    &program_id.to_bytes(),
    AoError::WrongEventQueueOwner,
)?;
check_account_owner(self.bids, &program_id.to_bytes(), AoError::WrongBidsOwner)?;
check_account_owner(self.asks, &program_id.to_bytes(), AoError::WrongAsksOwner)?;
```

The instruction implicitly checks ownership when it attempts to overwrite the account buffer later, so this is not exploitable. However, it is good practice to perform all such checks in advance.

Remediation

Aob's `create_market` instruction should explicitly check the `market` account's owner.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-08: Redundant account checks

Description

In Aob's `consume_events` instruction, the market's event queue is validated twice.

```
// agnostic-orderbook/program/src/processor/consume_events.rs:L133-L148

check_account_key(
    accounts.event_queue,
    &market_state.event_queue,
    AoError::WrongEventQueueAccount,
)?;
...
check_account_key(
    accounts.event_queue,
    &market_state.event_queue,
    AoError::WrongEventQueueAccount,
)?;
```

In Aob's `new_order` instruction, the event queue header's account tag is checked twice: both in its `process` function and the `EventQueue::new_safe` static method.

```
// agnostic-orderbook/program/src/processor/new_order.rs:L139-L146

let header = {
    let mut event_queue_data: &[u8] =
        &accounts.event_queue.data.borrow()[0..EVENT_QUEUE_HEADER_LEN];
    EventQueueHeader::deserialize(&mut event_queue_data)
        .unwrap()
        .check()?
};
let mut event_queue = EventQueue::new_safe(header, accounts.event_queue,
callback_info_len)?;
```

In Dex's `new_order` instruction, the user's owner and market are validated twice.

```
// dex-v4/program/src/processor/new_order.rs:L186-L201

if user_account.header.owner != self.user_owner.key.to_bytes() {
    msg!("Invalid user account owner provided!");
    return Err(ProgramError::InvalidArgument);
}
if user_account.header.market != self.market.key.to_bytes() {
    msg!("The provided user account doesn't match the current market");
```

© 2022 OtterSec LLC. All Rights Reserved.

```
    return Err(ProgramError::InvalidArgument);
};
if user_account.header.owner != self.user_owner.key.to_bytes() {
    msg!("Invalid user account owner provided!");
    return Err(ProgramError::InvalidArgument);
}
if user_account.header.market != self.market.key.to_bytes() {
    msg!("The provided user account doesn't match the current market");
    return Err(ProgramError::InvalidArgument);
};
```

Remediation

Remove the redundant account checks.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-09: Outdated Cargo dependency

Description

We performed our audit against commit 176b410 of Aob and commit 748aaf9 of Dex. However, this Dex commit uses an old version of Aob in `Cargo.lock`, namely commit 422b943.

```
// dex-v4/program/Cargo.lock:L57-L60

[[package]]
name = "agnostic-orderbook"
version = "0.1.0"
source =
"git+https://github.com/Bonfida/agnostic-orderbook.git#482b943297e9c02ee9aa05858223
ef5d8386d78a"
```

A number of bugs, fixed in later Aob commits, remained in Dex nonetheless.

Remediation

To avoid outdated dependencies, run `cargo update`.

© 2022 OtterSec LLC. All Rights Reserved.

OS-SRM-SUG-10: Market creation allows duplicate accounts

Description

The same account address may be repeatedly specified when invoking Aob's `create_market` instruction. Although the instruction succeeds, this results in a dysfunctional market.

The issue is that all checks, in particular that accounts are uninitialized, occur at the instruction's start. This does not prevent a buffer from being repeatedly overwritten, e.g. as an event queue and then a bids slab.

// agnostic-orderbook/program/src/processor/consume_events.rs:L133-L148

```
check_unitialized(accounts.event_queue)?;
check_unitialized(accounts.bids)?;
check_unitialized(accounts.asks)?;
check_unitialized(accounts.market)?;
```

Remediation

Each call to `utils::check_unitialized` should be made immediately before the corresponding account is initialized.

© 2022 OtterSec LLC. All Rights Reserved.

07 | Appendix

Appendix A: Program Files

Listed below are the files in scope for this audit and their truncated sha256 hashes.

agnostic-orderbook

```
Cargo.lock                          83f698b6f4260f972a041b8ed83bb6b8
Cargo.toml                          8a409066b7c8c920a39f14f3bcdffba4
Xargo.toml                          a4292e0c9687ede7dc40c108682afb35
src
  critbit.rs                        3cda49b8b961f769cdd562dc6593540f
  entrypoint.rs                     9ae83c3a9e7e86c4f9d1fdbc48ec31cf
  error.rs                          7cbe15d6aef0456faadc516fae4cb487
  instruction.rs                    a0c229adf296e615dbc1ffe8d5f050bc
  lib.rs                            4a044659629e682f465f5c7f515c61ac
  orderbook.rs                      4c57f1410e4f99edfe6d2e4dd8311be6
  processor.rs                      1c9bab4431c6542bf08c6f621fd46e7b
  state.rs                          27206f51ebca6b615bcce3dfde7b6bd4
  utils.rs                          eb2e07c3ed56345b905c99aab567df46
  processor
    cancel_order.rs                 f3cf48768d901e4ab976182904d81a92
    close_market.rs                 836eedf02b7950503c3878f9e2b35457
    consume_events.rs               99db4889f4bee4ce69e922200b4181a7
    create_market.rs                c6493ec651a71f99b27b9f92f3149d22
    new_order.rs                    cdac1566cbffcabcdbedf876f53407c3
tests
  functional.rs                     7b642756244885d3b641c2561d44138a
  common
    mod.rs                          3da9e41e54128210d823f125f1da814f
    utils.rs                        9472fa8cbdf6e03c707301128d20dc57
```

dex-v4

```
Cargo.lock                          310e4f149effdcedb20d178a656e3ce7
Cargo.toml                          a7a08d54bd286919a944a94558715c76
Xargo.toml                          a4292e0c9687ede7dc40c108682afb35
src
  entrypoint.rs                     bcf756728cdac05a8716b0d6b977e4c4
  error.rs                          48ae020d545f67fc8ef96e588f08aac9
  instruction.rs                    59c0396252677fcda665dae41a4f0f54
  instruction_auto.rs               7901e4f65f80295290ef3bc6a65981a4
  lib.rs                            90f25d92a7fb3b517788945cf6ec54c9
  processor.rs                      6696790e9c919814511aa84ec2e5903b
  state.rs                          3693aeca7f9b68d8f7837fb5c54ea81b
```

© 2022 OtterSec LLC. All Rights Reserved.

```
  utils.rs                          a8343159034b09b952556827f55e10f9
  processor
    cancel_order.rs                 5b323780fccc9793f099726d5873e239
    close_account.rs                de3615dfbfea5be296016bd934884b7f
    close_market.rs                 eea6d8044b88a47734defe45425f1813
    consume_events.rs               189e586f1aabdea787994844c6c88d48
    create_market.rs                414adb04f6da8a71cf1e9e73c977f3f9
    initialize_account.rs           458c07e4be60c97df9e5b3e15c396e1c
    new_order.rs                    084500effd057f8fb55fb1bb49bf9783
    settle.rs                       0953554bf666a88d89775e637695e6d7
    swap.rs                         28dc391dd385e7186d1fdf02fc6ac249
    sweep_fees.rs                   b4ea33d6135f3c086a85211671524357
tests
  functional.rs                     6aee725730b8b542af1f6581a35d25f3
  performance_tests.rs              14402fb6fa079572551133f3b4a73aeb
  common
    mod.rs                          bbc587cc2114acf264591c197ecd6c0b
    performance_test_utils.rs       af9f8687b227519f48920043104964a7
    utils.rs                        a730893a9a409ef94089c23663226f8e
    performance_tests_parse
      parse_log.py                  370e8d4a1d949606afb8de93fa9dc8f8
      run_perf_test.sh              9722a40a1c31cd7d0e5f72b37e75541c
```

© 2022 OtterSec LLC. All Rights Reserved.

Appendix B: Implementation Security Checklist

Unsafe arithmetic

Integer underflows or overflows	Unconstrained input sizes could lead to integer over or underflows, causing potentially unexpected behavior. Ensure that for unchecked arithmetic, all integers are properly bound.
Rounding	Rounding should always be done against the user to avoid potentially exploitable off-by-one vulnerabilities.
Conversions	Rust `as` conversions can cause truncation if the source value does not fit into the destination type. While this is not undefined behavior, such truncation could still lead to unexpected behavior by the program.

Account security

Account Ownership	Account ownership should be properly checked to avoid type confusion attacks. For Anchor, the safety of unchecked accounts should be clearly justified and immediately obvious.
Accounts	For non-Anchor programs, the type of the account should be explicitly validated to avoid type confusion attacks.
Signer Checks	Privileged operations should ensure that the operation is signed by the correct accounts.
PDA Seeds	PDA seeds are uniquely chosen to differentiate between different object classes, avoiding collision.

Input validation

Timestamps	Timestamp inputs should be properly validated against the current clock time. Timestamps which are meant to be in the future should be explicitly validated so.
Numbers	Sane limits should be put on numerical input data to mitigate the risk of unexpected over and underflows. Input data should be constrained to the smallest size type possible, and upcasted for unchecked arithmetic.
Strings	Strings should have reasonable size restrictions to prevent denial of service conditions.

© 2022 OtterSec LLC. All Rights Reserved.

Internal State	If there is internal state, ensure that there is explicit validation on the input account's state before engaging in any state transitions. For example, only open accounts should be eligible for closing.

Miscellaneous

Libraries	Out of date libraries should not include any publicly disclosed vulnerabilities.
Clippy	`cargo clippy` is an effective linter to detect potential anti-practices.

© 2022 OtterSec LLC. All Rights Reserved.

Appendix C: Proofs of Concept

Listed below are the provided proof of concept files and their truncated sha256 hashes.

```
run.sh                                  6af5799eecc52c2491096ae738f1f542
os-srm-adv-00
  patch                                 8988ed2148db86028c669ec3da121533
os-srm-adv-01
  patch                                 756c7dae7370a063df6ccc0e7678005a
os-srm-adv-02
  patch                                 70a072543a0a838d4bb4434b82dfdd8f
os-srm-adv-03
  patch                                 06ce4041c75f75c1e5b0cc19a60900b4
os-srm-adv-04
  patch                                 f1b136cab6e42a450fd023ef2acaa508
os-srm-adv-05
  patch                                 5cc17ba41bad3526436620a24d3f0451
os-srm-adv-06
  patch                                 e6f3ab972fe4a1cc9ff432812f633bde
os-srm-adv-07
  patch                                 21a4b4ba7a5602c7f69271f601198a93
```

© 2022 OtterSec LLC. All Rights Reserved.

Appendix D: Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities which **immediately** lead to loss of user funds with minimal preconditions. • Misconfigured authority/token account validation • Rounding errors on token transfers
High	Vulnerabilities which **could** lead to loss of user funds but are potentially difficult to exploit. • Loss of funds requiring specific victim interactions • Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities which could lead to denial of service scenarios or degraded usability. • Computation limit exhaustion due to malicious input • Forced exceptions preventing normal use
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. • Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as *general findings*. • Explicit assertion of critical internal invariants • Improved input validation • Uncaught Rust errors (vector out of bounds indexing)

© 2022 OtterSec LLC. All Rights Reserved.